TrustTier: Cost-Tiered Zero-Knowledge Verification of DeFi Liquidation Decisions on Verified On-Chain Inputs

CSD436 Blockchain Technology — Final Project Paper

Gaurav Rai
Shiv Nadar University
Delhi NCR, India
2410110133

Shaurya Sharma
Shiv Nadar University
Delhi NCR, India
2410110317

Sharanya Gupta
Shiv Nadar University
Delhi NCR, India
2410110315

ABSTRACT

Lending protocols liquidate a borrower only once the loan-to-value ratio (LTV) crosses a threshold, which is free to check but acts too late in a crash; acting earlier needs a risk model, and a model's score is usually computed by an operator nobody can audit. Zero-knowledge machine learning (zkML) can prove that a score came from a committed model, but a proof-verified decision costs about 0.9 M gas, too much to spend on every position. TrustTier makes the strength of verification grow with risk. A position far from its liquidation boundary may be decided by a bonded heuristic claim (Tier 0), one approaching it by a bonded, salted commitment to a registered model's output (Tier 1), and one at the boundary only by an on-chain-verified Halo2 zk-SNARK of the model's inference (Tier 2). Cheap claims stay accountable through a one-hour window in which anyone can slash a false claim with a Tier-2 proof; the bond scales with the base fee so that slashing always pays, challenges are committed before they are revealed so they cannot be front-run, and a Tier-1 claimant must open its commitment or forfeit. A verified Tier-2 approval is the only way the lending protocol's liquidator can execute a (soft) liquidation, once, within 30 minutes.

A tiered design is only as honest as whoever picks the tier and supplies the model's inputs. TrustTier therefore trusts neither: the contract reads every input itself from the lending protocol and Chainlink-style feeds, computes the distance δ to the boundary, refuses a tier too weak for it, and accepts a proof only if its public inputs equal the features it computed in the same transaction. Cheap claims snapshot their features, so a challenge is judged on the facts at claim time.

The prototype comprises nine Solidity contracts (including the liquidator that consumes approvals, an Aave V3 adapter and multisig-plus-timelock governance), a router, an EZKL proving pipeline, a sentinel, a SQLite ledger indexed from the chain, a one-click Sepolia deployment, and a DApp in which users generate their own Ethereum accounts and sign their own transactions; 126 contract tests and 92 Python tests pass. On 374 positions (74 real Aave V2 snapshots and 300 synthetic), tiering costs 210.3 M gas against 328.4 M for always proving, a 36.0% saving; a sweep of 34 band settings shows the deployed bands are Pareto-optimal. In a 30-day stress test of 3,000 positions on identical price paths, acting on the proven model cuts forced liquidations by 88% and insolvencies from 61 to 2 compared with the health-factor rule, at the price of decision gas and 26% unnecessary soft liquidations; the gas makes daily monitoring affordable for retail positions on a rollup, not on L1. Opcode traces of whole blocks show that 374 decisions parallelize to 15.3× on 16 threads, that bonds paid as `msg.value` would cap this at 3.33× on engines that treat balance updates as read-modify-writes, and that deposit-funded bonds remove that bottleneck.

KEYWORDS

DeFi, liquidations, zkML, zk-SNARK, EZKL, Halo2, smart contracts, optimistic verification, oracles, parallel execution

1 INTRODUCTION

Lending protocols such as Aave, Compound and MakerDAO hold billions of dollars of collateral, and a liquidation sells part of it at a discount. In the four largest protocols, 28,138 liquidations sold USD 807.46 M of collateral [28]. Whether a position is truly at risk is increasingly judged by a model that looks beyond the LTV, at volatility, price momentum and account history. Today that model runs on an operator's server. Borrowers, liquidators and auditors see only the outcome, and must trust that the score came from the claimed model, on the real inputs, and that neither the model nor the record was quietly changed.

Why a model at all. Every protocol already has a free, deterministic rule: liquidate once LTV reaches the threshold (Aave's health factor below 1). It acts only at the boundary, so a crash day can carry a position from safe to under water before any keeper moves, and the protocol absorbs bad debt. A model that sees volatility and momentum can act *before* the boundary with a smaller, *soft* liquidation. That is only acceptable if the model's verdict is verifiable, and only affordable if verification is not paid on every position. Section 7.8 measures this trade against the health-factor rule on identical price paths.

Recent work shows that such decisions *can* be verified on-chain, either by running a small model inside a contract [2] or by verifying a zk-SNARK of its inference [10, 20]. The obstacle is cost. In our implementation a proof-verified liquidation decision costs 878,059 gas and about 16 s of proving, 3.3 times a bonded heuristic decision. Spending that on every position is wasteful, because most positions are nowhere near their liquidation boundary.

The idea. TrustTier spends verification where it matters. It borrows the intuition of memory tiering, where hot data gets the expensive fast tier: positions far from the boundary get cheap, bonded claims, and only positions at the boundary must carry a zero-knowledge proof. The cheap tiers are optimistic, as in optimistic rollups [19] and optimistic ML [11], but a dispute is settled by a single zk-SNARK rather than an interactive bisection game.

The catch, and our answer. Tiering creates a new trust problem. Our midterm prototype let an off-chain router compute δ and choose the tier, and a later version let a "feature oracle" commit each position's inputs. A reviewer put the problem precisely: *the router is an unaudited operator, and the chain cannot check δ.* Whoever supplies the inputs controls which tier applies and what the proof proves. This paper's central design change removes that party entirely. The contract reads collateral, debt and the liquidation threshold from the lending protocol, volatility and 24-hour momentum from Chainlink feeds, and account history from protocol timestamps; it quantizes them exactly as the circuit does, enforces the tier from its own δ, and requires the proof's public inputs to equal its own features. We call this *verified inputs instead of trusted inputs.*

Contributions.

- A risk-proportional verification scheme with three tiers whose boundaries are enforced *on-chain*, from inputs the contract reads itself (Sections 5.1–5.2).
- A bonded challenge window with feature snapshots, a precise position lifecycle in which a Tier-2 proof supersedes any pending cheap claim, and deposit-funded bonds (Sections 5.3–5.8).
- An incentive and execution layer: gas-scaled bonds that make slashing profitable at any gas price, a protocol-funded reward pool for honest claims, Tier-1 commitments that must be opened, commit-reveal challenges, feature anchors for slow proofs, and a liquidator contract that executes each Tier-2 approval exactly once (Sections 5.5–5.6).
- A complete implementation: nine contracts, an EZKL/Halo2 circuit with a 15.6 KB verifier, an Aave V3 adapter, multisig-plus-timelock governance, a router, a sentinel and a DApp (Section 6).
- An evaluation with real proofs and measured gas: a comparison with the health-factor rule on identical price paths, the economics of the challenge game and of deployment on L1 versus a rollup, an outcome-labelled test on real Aave data, tiering cost, ablations of every design component, a 34-point band sweep, a threshold sweep, model calibration, and three concurrency studies (modelled scheduling, opcode-traced blocks executed on 1–64 simulated threads, and competing liquidator threads on a live node) (Section 7).
- A security analysis of 31 findings, each with its status (Section 8).

Scope. TrustTier governs the *decision* that a liquidation is justified and gates its execution: in the demo lending pool, only `TrustTierLiquidator`, and only on an open Tier-2 approval, can liquidate. It does not replace a protocol's own health-factor liquidations, which remain the fallback. The prototype runs on a local Ethereum chain (Hardhat) with a tested deployment path to the Sepolia testnet.

2 BACKGROUND AND MOTIVATION

Liquidations. A lending position is liquidatable when its LTV exceeds the market's liquidation threshold θ. Liquidation designs regularly sell more collateral than needed; Aave V2 had accumulated up to USD 87.4 K of bad debt by April 2021; and 73.97% of liquidations paid an above-average transaction fee because liquidator bots race each other [28]. Small price moves can make large volumes of positions liquidatable at once [26]. Decisions near the boundary are therefore both the most contested and the most consequential.

zkML.. A zk-SNARK lets a prover convince a verifier that a computation produced an output without revealing private inputs [15, 18]. EZKL [34] compiles an ONNX model into a Halo2 (PLONKish) circuit with KZG polynomial commitments [21] and emits a Solidity verifier; the proof is small and checked by a pairing computation. Proving cost grows with model size [10, 25], which is why TrustTier uses a deliberately small 5⟶8⟶1 network.

Three observations. (1) *Risk is concentrated*: in our 374-position evaluation set only 144 positions (38.5%) are within $\delta \leq 0.05$ of their threshold, so proving every position wastes most of the cost. (2) *Blockchains already provide the right primitives*: immutable logs give non-repudiation, consensus order resolves races between bots, contract-held bonds make cheap claims costly to fake, and Chainlink-style feeds and protocol state are readable on-chain, so inputs need not be trusted. (3) *Liquidations come in bursts*: when prices fall, many positions cross the boundary within the same few blocks, so the contracts should not stop a parallel execution engine from processing those decisions concurrently [17].

3 RELATED WORK

Prior work verifies ML decisions on-chain either completely, running or proving every inference, or optimistically, checking only disputes. None adapts the strength of verification to the risk of the individual decision, and none removes trust in the inputs of a tiered scheme. Table 1 compares the closest work.

DeFi liquidations and attacks. Qin et al. [28] and Perez et al. [26] measure liquidation incentives, over-liquidation and instability. Zhou et al. [33] catalogue DeFi attacks, including oracle manipulation, which in TrustTier could move a position between tiers; we therefore add staleness checks and leave manipulation resistance to the oracle (Section 9). Daian et al. [12] and Qin, Zhou and Gervais [29] show that the public mempool lets bots front-run and reorder transactions, so liquidators race for the same position. TrustTier admits one decision per position and makes losing attempts revert cheaply (Section 7.11).

Concurrent execution of smart contracts. Dickerson et al. [13] introduced speculative execution by miners, who publish a schedule that validators replay. Anjana et al. [3] execute a block with optimistic software transactional memory and ship a block graph for parallel validation; OptSmart [4] reduces its space cost. Saraph and Herlihy [31] measured on historical Ethereum blocks how much speculative execution helps, and found that conflicts concentrate on a small amount of popular state. Block-STM [17] runs in production with multi-version memory and a preset order, ParallelEVM [22] detects conflicts at the level of individual operations, and Conthereum [32] schedules conflicting transactions in advance. BlockRaFT [27] scales a single blockchain node out by running it as a Raft-coordinated cluster whose leader hands work to followers; TrustTier's router borrows that leader/worker shape for its own bottleneck, Tier-2 proving (Section 7.12). All of them gain only when transactions touch disjoint state, which is why TrustTier keys

Table 1: Verifiable and on-chain ML: what each work does and the gap TrustTier addresses.

Work	Problem	Approach	Key result	Gap TrustTier addresses
Alhaidari et al., AFT 2025 [2] (primary baseline)	Off-chain detectors miss DeFi exploits executed within one block	Train models on Layer 2 under a Proof-of-Improvement protocol; run quantized models directly in an L1 contract	Over 97% detection on 298 real exploits; 57,603–506,397 gas per inference	Weights and inputs become public; TrustTier keeps the model private, proves only where needed, and verifies its inputs on-chain
Chen et al., EuroSys 2024 [10]; Kang et al., 2022 [20]	Zero-knowledge proofs of ML inference are slow	Compile ML models to optimized Halo2 circuits	Practical proofs for realistic models; cost grows with model size	A proof on every decision is still costly; TrustTier proves only near the boundary
Garg et al., CCS 2023 [16]	Proving how a model was trained	Zero-knowledge proofs of training	Feasible for simple models such as logistic regression	Model provenance can be proved; TrustTier binds every decision to a registered model hash
Peng et al., 2025 [25]	Fragmented landscape of verifiable ML	Survey of ZKP-based verifiable training, inference and testing	Proving cost is the main barrier	Justifies the deliberately tiny model
Conway et al., 2024 [11]; Kalodner et al., 2018 [19]	Full verification is too heavy	Optimistic execution: results accepted unless disputed via interactive bisection	Low cost; finality after a challenge period	TrustTier keeps the bonded window but settles disputes with one zk-SNARK, judged on an on-chain feature snapshot

every write by position, account or model, and why we trace real read/write sets rather than assume them.

Contract security and proof systems. Atzei et al. [5] classify attacks on Ethereum contracts, including reentrancy and failed sends; TrustTier answers them with checks-effects-interactions and pull payments. KZG commitments [21] need a trusted setup; zk-STARKs [6] remove it at the price of much larger proofs [23], a trade-off Section 5.9 quantifies.

4 THREAT MODEL AND GOALS

Actors. Borrowers own positions in a lending protocol. *Liquidators* (people or bots) submit decisions and may be dishonest: they may claim a cheaper tier than a position deserves, submit forged or replayed proofs, front-run each other, or try to lock a position to delay its liquidation. The *router* is our reference liquidator and gets no special rights. *Watchers* (any user of the DApp, or the sentinel) challenge false claims. *Governance* is an M-of-N multisig behind a timelock.

Trust assumptions. We trust exactly what the lending protocol already trusts: its own accounting (collateral, debt, threshold) and its price and volatility feeds, which must be fresh. We trust the soundness of Halo2 with KZG over BN254 under a correctly generated structured reference string (SRS), and that governance changes are visible a full delay before they apply. We do *not* trust the router, any liquidator, or any off-chain computation of features or δ.

Goals.

G1 Input integrity. Every decision is made on features derived from on-chain state; no party can substitute its own.

G2 Tier soundness. No decision uses a tier weaker than the position's on-chain δ allows.

G3 Proof binding. A Tier-2 proof approves a liquidation only for the position, and the state, it was made on, and only if it proves high risk.

G4 Accountability. A false cheap claim can be slashed; an honest one cannot.

G5 Liveness. No claim can delay a needed liquidation, and no position can be locked forever.

G6 Exactly once. Racing liquidators finalize a position at most once, and a race over one position never delays a decision about another.

G7 Governed change. No single key can change what the system trusts.

G8 Incentive compatibility. Slashing a false claim pays for itself at any gas price, a claimant cannot keep its committed score hidden, and a challenge seen in the mempool cannot be stolen.

5 DESIGN

5.1 Tiers

TrustTier sets the evidence a decision must carry by the position's distance from its boundary, $\delta = \theta - \text{LTV}$, where θ is the position's own liquidation threshold (Figure 1, Table 2). Tier 0 requires $\delta > B_0 = 0.15$, Tier 1 requires $\delta > B_1 = 0.05$, and Tier 2 is always allowed: a stronger tier than necessary is never refused. Section 7.6 justifies both band values.

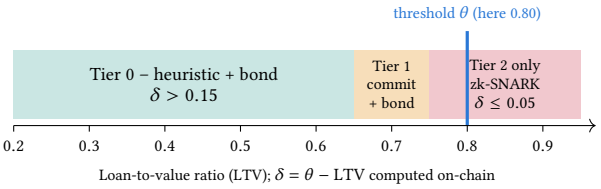


Figure 1: Verification tiers as bands of LTV for a market with $\theta = 0.80$. The contract reads θ per position and refuses a tier whose band the position is not in.

Table 2: Tiers, what each puts on-chain, and measured gas per decision (full transaction, including reading the position, two feeds and the 24-hour price round). *Claim 340,072 plus the mandatory reveal, 39,089.

Tier	Allowed	Evidence on-chain	Gas
0 Heuristic	$\delta > 0.15$	Decision record, feature snapshot, gas-scaled bond slashable for 1 h	269,534
1 Commitment	$\delta > 0.05$	Registered model, salted commitment to (features, score) that must be opened in the window, snapshot, same bond	379,161*
2 zk-SNARK	always	Halo2 proof that the model on the position's on-chain features outputs "high risk"; approves one liquidation	878,059

Table 3: The five features, their on-chain sources, and the exact integer quantization $q \in [0, 1024]$ the contract and the circuit share (C, D collateral and debt; p_t, p_{t-24h} price now and 24 h ago; v volatility feed answer in percent with d decimals; integer division throughout).

Feature (bounds)	On-chain source	Quantization
LTV [0, 1.5]	lending protocol	$\min(2048D/3C,\ 1024)$
Volatility [0, 1]	Chainlink realized-vol feed	$\min(1024v/(100 \cdot 10^d),\ 1024)$
Momentum [−1, 1]	Chainlink price, round 24 h ago	$\min(512\, p_t/p_{t-24h},\ 1024)$
Days since top-up [0, 90]	protocol / adapter timestamp	$1024 \min(\lfloor \Delta t \rfloor_d, 90)/90$
Account age [0, 2000]	protocol / adapter timestamp	$1024 \min(\lfloor \Delta t \rfloor_d, 2000)/2000$

5.2 Verified inputs instead of trusted inputs

Where every feature comes from. The engine's view function `currentFeatures(positionId, pastRoundId)` computes the model's five features from on-chain sources only (Table 3). Positions are read through a small interface, `IPositionSource`, which returns collateral and debt (USD with 8 decimals), the liquidation threshold, the opening and last top-up times, and the market's two feeds. We implement it twice: `AaveV3PositionSource` reads Aave V3's `getUserAccountData` [1], and `MockLendingPool` values demo positions with Chainlink-compatible feeds.

Each feature is min-max normalized to $[0, 1]$ over declared bounds and scaled by 2^{10}, the circuit's input scale, so the integer q is exactly what the circuit sees ($q/1024$). Out-of-range values are clamped, never rejected, so no position becomes undecidable; a timestamp in the future counts as zero days. A Python mirror (`onchain_features.py`) reproduces the arithmetic bit for bit, and tests check it against the contract and against the public inputs of real proofs.

The 24-hour round. Momentum needs the price that was in effect 24 hours ago, which a contract cannot search for cheaply. The caller passes `pastRoundId`; the contract checks that this round was updated at or before $t - 24$h and that the next round (if any) was updated after it, so exactly one round is acceptable. The router finds it with an exponential walk back from the latest round followed by binary search over round timestamps. Chainlink proxy round IDs encode a phase in their upper bits and are consecutive only within a phase [8], so the search stays within the current phase. The last round of a phase has no successor; the contract then accepts it only if it was updated within one heartbeat (`maxPriceAge`) of the target, which an older, superseded round cannot be. A wrong round can only make the transaction revert (`BadMomentumRound`), never mis-price it. Both feeds must also be fresh (`maxPriceAge`, `maxVolatilityAge`; 3 h and 48 h by default) and positive, and on an L2 an optional Chainlink sequencer-uptime feed must report the sequencer up for at least an hour.

Tier enforcement. Every `decide*` call computes δ in basis points from the protocol's own collateral, debt and threshold and reverts with `TierNotAllowed(positionId, tier, deltaBps)` if the requested tier is too weak. A refused attempt does not consume the position. The router uses the same thresholds (a test reads them from the Solidity source), so it never attempts a tier the chain would refuse; it may choose a stronger tier, which only costs it gas.

Public features, private model. The circuit's five feature inputs are *public* instances and the model weights are private. The proof's instances are $[q_1, \ldots, q_5,\ \ell,\ \lfloor 2^{10} P(\text{low}) \rfloor,\ \lfloor 2^{10} P(\text{high}) \rfloor]$ with label $\ell \in \{0, 1\}$. `decideZkProof` requires instances$[0..4]$ to equal the q it computes in the same transaction, $\ell = 1$, the cited model to be both registered and the one the verifier encodes, and the verifier to accept. A proof made for another position, or for this position before its price, collateral or debt changed, fails with `ProofNotForPosition`. Publishing the features costs no privacy, because they are public chain data already.

5.3 Bonded challenge window with feature snapshots

Tier 0/1 decisions are optimistic. The claimant posts a bond (Section 5.5), which opens a one-hour window. The contract stores snap = keccak256$(q_1, \ldots, q_5)$, the features the claim was made on, and emits them in `FeaturesSnapshotted`. Anyone holding a valid Tier-2 proof whose features hash to snap and whose label is "high risk" can call `challengeDecision` within the window and is credited the bond. Otherwise `reclaimBond` credits it back to the claimant after the window. Because a zk-SNARK proves the claim wrong in one transaction, no interactive bisection game is needed.

Judging the challenge on the snapshot gives the two properties goal G4 asks for. An *honest* claimant is not slashed because prices moved after the claim: a proof on later features does not match the snapshot. A *false* claimant is slashed even if they top up the position afterwards: the snapshot still records the facts at claim time. Both are tested with real proofs. Payouts go to `pendingWithdrawals` and

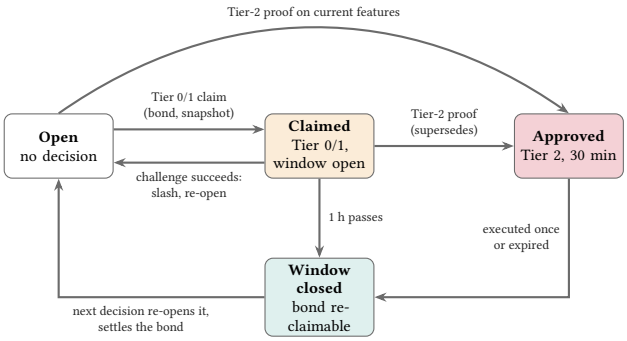


Figure 2: Position lifecycle. Cheap claims block only other cheap claims for one window; a Tier-2 proof of current high risk always proceeds and approves one liquidation, executed by the liquidator within 30 minutes or expired.

are pulled with `withdraw`, so a hostile recipient can only strand its own funds.

The Tier-1 commitment is keccak256(q, score $\cdot\ 10^6$, salt) with a random 32-byte salt, so the score cannot be brute-forced from the hash while the claim is fresh. The claimant must open it with `revealCommitment` inside the window: the contract recomputes the hash and checks that q is the snapshot. A committed score of 0.5 or more contradicts the claim itself, so the bond is forfeited and the position re-opens; an unopened commitment is forfeited when the window closes. Tier 1 is therefore stronger than Tier 0 on-chain, not only for an auditor: the claimant is bound to a specific score from a registered model and must publish it.

5.4 Position lifecycle

Figure 2 shows the states a position moves through. Three rules give goals G5 and G6.

- **One cheap claim at a time.** While a claim's window is open, further Tier 0/1 claims revert (`PositionAlreadyFinalized`). The lock is the first thing every `decide*` checks, before any external call, so a losing racer pays only ≈26 k gas.
- **Tier 2 supersedes.** A Tier-2 proof that the position is high risk *now* is accepted even while a cheap claim's window is open (`ClaimSuperseded`). The superseded claim keeps its bond, snapshot and window: it is still slashed if it was false when made, and reclaimed if not. A claim therefore cannot delay a needed liquidation.
- **Re-opening.** A successful challenge re-opens the position so the challenger's proof can liquidate it. After an unchallenged window, the next decision re-opens it and settles the old bond. A Tier-2 approval blocks every other decision until the liquidator executes it or `APPROVAL_TTL` (30 minutes) passes; either way the position, partly repaid or not, can then be decided again.

5.5 Incentives

An optimistic tier is secure only if someone checks it. A watcher slashes a false claim only when the bond it wins exceeds the challenge's cost, about 0.68 M gas: with a fixed 0.01 ETH bond that stops being true above 14.7 gwei (Section 7.9), so at ordinary L1 gas prices nobody has a reason to watch, the verifier's dilemma. TrustTier closes the gap in four ways.

- **Gas-scaled bonds.** A claim must post max(0.01 ETH, 1.5 M · basefee), read from `block.basefee` at the claim. The bond covers a challenge at any gas price and stays profitable while the base fee at challenge time is below 2.2× the base fee at the claim.
- **Rewards for honest work.** The lending protocol funds a reward pool; an unchallenged claim earns `claimReward` when its bond is settled, at most once per position per day, so attesting to safe positions is paid work and cannot be farmed by re-claiming.
- **Commit-reveal challenges.** A challenger first commits keccak256(challenger, position, snap, keccak256(proof)) and reveals the proof at least one block later. Once any commit exists for a claim, the one-step `challengeDecision` is closed for it, so a bot that copies a revealed proof from the mempool has no commit and cannot claim the bond.
- **Must-open commitments** (Section 5.3) make a Tier-1 claim self-checking.

5.6 Executing decisions

A decision nobody acts on is only a record. `TrustTierLiquidator` is the lending protocol's one path to liquidation: `liquidate(position)` calls `consumeApproval`, which only the configured liquidator may call and which succeeds only on an open, unexpired Tier-2 approval, then asks the protocol to execute. The demo pool performs a *soft* liquidation: it closes 50% of the debt and moves collateral worth the repaid debt plus a 5% bonus to whoever called, which pays the keeper for the proof. One proof buys exactly one liquidation. Because the model is proven on-chain, the pool may act *before* LTV reaches the threshold, which is the reason to use a model at all; the protocol's own health-factor liquidation stays in force as a fallback.

Slow proofs. A Tier-2 proof takes about 16 s, longer than a block, and a feed update in between changes q and invalidates it. Anyone may `anchorFeatures(position)`, which stores the hash of the current q and δ. For 10 minutes a proof on the anchored features is accepted, but only while the position has not become safer than at the anchor ($\delta_{\text{now}} \le \delta_{\text{anchor}}$). The router anchors only when a feed moved during proving, and then proves once more.

5.7 Governance and model registry

`ModelRegistry` holds the set of trusted model hashes; a revoked hash cannot be revoked again or silently re-registered. `setTier2Verifier(verifier, modelHash)` binds the verifier to the one model it encodes, and Tier-2 decisions must cite exactly that hash. All owner functions (registering or revoking models, swapping the verifier, changing the position source or feed staleness limits) belong to an OpenZeppelin `TimelockController` [24], whose only proposer and executor is an M-of-N multisig. Every change therefore needs M signatures and a public delay (goal G7).

5.8 Designing for concurrent execution

Parallel execution engines gain only when transactions touch disjoint state [17, 31]. TrustTier follows one rule: every storage write is keyed by a single position, account or model. The midterm audit

Table 4: SNARK vs STARK for Tier 2. TrustTier numbers are measured; STARK numbers are from the cited sources.

Criterion	Halo2/KZG (TrustTier)	STARK
Proof size	3,584 B	68,564 B vs 384 B Groth16 [23]; 166 KB Cairo [35]
On-chain verification	621,921 gas	≈9 M gas, amortized over a batch [14]
ML tooling	EZKL: ONNX → circuit → verifier	Less mature for small ONNX models
Trusted setup	Required (KZG SRS)	None
Post-quantum	No	Yes
Fits	One proof per decision	Batched settlement

log broke it by appending every decision to one array, whose length slot every transaction wrote; `TierAuditLog` now stores one record per position and history is read from events. Per-position engine state is packed into three slots (claimant and bond; deadline and three flags; snapshot), which also saves about 46 k gas per cheap claim.

Opcode traces of whole blocks (Section 7.11) exposed one more shared write that storage keys do not show: a bond paid as `msg.value` changes the engine's ETH balance. On an engine that treats a balance change as a read-modify-write, every cheap claim in a block then conflicts with every other. We therefore added *deposit-funded bonds*: `deposit()` pre-funds the claimant's own balance in `pendingWithdrawals`, a claim sent without ETH draws `MIN_BOND` from it, and reclaimed or slashed bonds are recycled the same way. Such a claim touches only its claimant's slot, for 5,238 extra gas. Rewards are paid when a bond is settled, not when a claim is made, so the reward pool's shared balance is never written by a claim.

5.9 Why a SNARK, not a STARK

zk-STARKs need no trusted setup and resist quantum attacks [6], so they are the natural alternative for Tier 2. The right choice depends on how proofs are used, and TrustTier verifies proofs one at a time, inside the liquidation that depends on them. Published measurements favour a SNARK for that pattern (Table 4). For the same statement, a STARK proof was 68,564 bytes against 384 bytes for Groth16, about 178 times larger [23]. Verifying one StarkWare proof on Ethereum costs about 9 M gas, affordable only because one proof covers a whole batch [14]. A Cairo proof of a tiny Fibonacci program is 166 KB [35]. Even STARK-based production systems wrap their STARK in a Groth16 proof for the on-chain step [30]. Our Halo2 proof is 3,584 bytes and verifies for 621,921 gas. It is larger than a Groth16 proof because Halo2/KZG uses a universal, circuit-independent setup, and EZKL turns an ONNX model into a circuit and verifier directly [34]. We accept a trusted setup and no post-quantum security; if Tier-2 decisions were batched into a periodic settlement proof, a STARK's economics would win.

6 IMPLEMENTATION

Figure 3 shows the system.

Contracts. `LiquidationEngine` (14,100 bytes deployed), `TrustTierLiquidator`, `ModelRegistry`, `TierAuditLog`, the EZKL-generated `Halo2Verifier` (15,612 bytes, under the 24,576-byte EIP-170 limit [7]), `AaveV3PositionSource`, `MockLendingPool`, `TrustTierMultisig` and `TrustTierTimelock`, in Solidity 0.8.24 with OpenZeppelin 5. Every revert is a custom error. EZKL's verifier reverts on a bad proof instead of returning false, so the engine wraps it in `try/catch`. The Aave adapter cannot read account age or top-up times from Aave, which stores neither; it records registration time and lets anyone `poke` an account to record a top-up when its aToken's *scaled* balance (deposits without accrued interest) grows. Both are derived from on-chain state only.

Risk model. A 5→8→1 multilayer perceptron (scikit-learn) on the five normalized features, exported to ONNX with logits clipped to ±8 inside the graph. Synthetic labels are centred near LTV 0.75, just below a 0.80 threshold. Validation accuracy is 95.9% and ROC-AUC 0.995. The model's keccak256 hash is what `ModelRegistry` stores.

Circuit. EZKL 23.0.5 with input and parameter scale 2^{10}, public inputs, private parameters and public outputs, 2^{16} rows, and one inner column (`num_inner_cols=1`), which keeps the verifier under the EIP-170 limit; a Poseidon-hashed input variant did not fit. EZKL's witness generation hangs on inputs outside its calibrated range, so we calibrate over the whole normalized domain including all corners, and check circuit/ONNX label agreement on 200 random feature vectors (0 disagreements). The 8.4 MB SRS is self-generated; EZKL's setup is deterministic for a given SRS, so the 575 MB proving key is not shipped and is rebuilt in 12 s, with a check that it matches the shipped verification key and verifier.

Router, sentinel and DApp. The FastAPI router reads the engine's own view of a position, picks the weakest tier the chain accepts, runs the model, and escalates a Tier-1 position to Tier 2 when the model flags it, since a cheap "safe" claim on it would be false and slashable. A watcher endpoint builds a Tier-2 proof on a claim's snapshot (read from `FeaturesSnapshotted`). The React DApp signs users in with Sign-In with Ethereum (ERC-4361 [9]); users open demo positions, see the contract's features, δ, allowed tier and the model's verdict, push mock prices, run a five-step guided demo, watch open claims and pending approvals with live countdowns, and challenge (commit, then reveal), settle bonds, execute liquidations and withdraw from their own wallets. Users can generate an Ethereum account in the browser (a BIP-39 phrase and a password-encrypted keystore that never leaves the device) or bring MetaMask; after signing in with Ethereum, the account signs its own claims, challenges, Tier-2 decisions (with a proof from the router) and liquidations. A SQLite ledger indexes every contract event with its sender and block time and records every action requested through the router; it is an index of the chain, rebuilt from it at any time, and a restarted local chain opens a new epoch instead of mixing runs. A GitHub Actions workflow deploys to Sepolia, verifies the sources on Etherscan and publishes the addresses; the hosted DApp then reads the contracts directly over a public RPC with no server. When no chain is reachable, it runs the contract's rules, the real model (exported weights, checked against ONNX) and the measured gas in the browser, clearly labelled as a simulation. The sentinel polls events and raises a critical alert when a challenge succeeds. There is no database: on-chain events are the history.

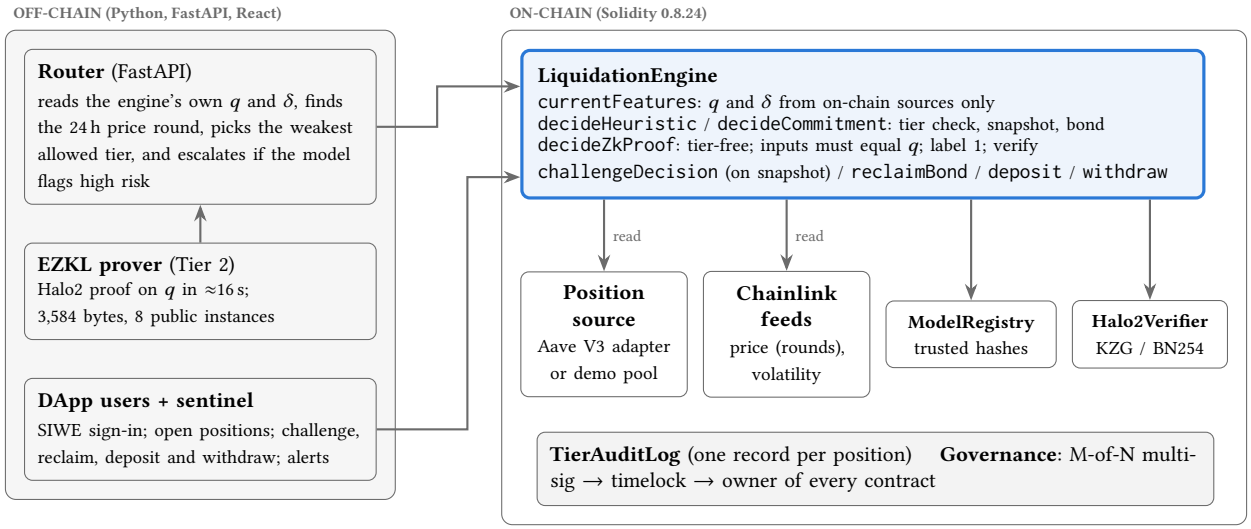


Figure 3: TrustTier architecture. The engine reads every input itself; the router only chooses a tier the engine will accept and supplies proofs. Users act from their own wallets.

Prover pool. Inspired by BlockRaFT [27], the router acts as a leader that hands Tier-2 proofs to workers: local processes, each with its own share of the cores, or prover workers on other machines reached over HTTP. When a batch of positions arrives (a crash), cheap claims are submitted at once while every proof runs in parallel; Tier-2 decisions are then submitted in order from the router's one account. Workers need no trust, since a wrong proof only makes the router's transaction revert.

Testing. 126 Hardhat tests cover every function and custom error, real proofs against the real verifier end to end, the Aave adapter, governance, tier enforcement, snapshots, lifecycle, deposits, gas-scaled bonds, rewards, reveals, commit-reveal challenges, anchors, the liquidator, the sequencer check and a Chainlink phase boundary. 92 Python tests cover the feature mirror against real proof instances, the tier rule, commitments, the 24-hour round search (against brute force on 50 random histories), the cost model, the parallel-execution simulator and the DApp's exported model.

7 EVALUATION

7.1 Setup

All measurements ran on one 2-vCPU Intel Xeon (2.8 GHz) machine with 7 GB of RAM: Hardhat 2.29 (in-process chain or local node), Node 22, Python 3.13, EZKL 23.0.5, scikit-learn 1.9, ONNX Runtime 1.29. Gas is measured by executing every entry point on a fresh chain (`scripts/measure_gas.js`), with the real verifier and real proofs, and includes the 21,000-gas transaction base. The evaluation set has 374 positions: 74 snapshots of one real Aave V2 wallet's health-factor history, including a genuine liquidation, with each snapshot's own threshold; and 300 synthetic positions spread across safe, watch and critical LTV bands with a 0.80 threshold. Cost results apply the measured per-tier gas to this set; re-proving hundreds of positions would change nothing, because proving time and verification gas depend on the fixed circuit, not on the input.

Table 5: Tier-2 proving pipeline (measured).

Circuit rows / input scale	2^{16} / 2^{10}
Public instances	8 (5 features, label, 2 probabilities)
Proving time (3 samples; 4 router runs)	16.0–16.5 s; 15.5–16.2 s
Proving key / verification key / SRS	575 MB / 141 KB / 8.4 MB
Proving-key rebuild from SRS	12.1 s
Proof calldata	3,584 bytes
Verifier: deployed size / deployment gas	15,612 B / 3,422,602
`verifyProof` alone / full `decideZkProof`	621,921 / 878,059 gas
Circuit vs ONNX label agreement	200 / 200

7.2 Proving and verification

Table 5 summarizes the Tier-2 pipeline. Three sample positions are defined by on-chain parameters (a high-risk position at LTV 0.93, a low-risk one at 0.20, and a watch-band position at 0.72 that the model flags) and proved for real. The contract's computed q equals each proof's first five instances.

7.3 Gas of every entry point

Table 6 lists every entry point. `currentFeatures`, called as a view, costs 92,510 gas including the base cost. It is the price of verified inputs and is included in every `decide*` row. A challenge with a real proof costs 678,771 gas, below a full Tier-2 decision because it checks a stored snapshot instead of recomputing features; committing first adds 94,771. Executing an approval costs 96,402 gas.

7.4 Cost of tiering

Table 7 applies the measured gas to the evaluation set, charging every Tier-1 claim its mandatory reveal. Tiering spends 210.3 M gas where proving every position would spend 328.4 M, a 36.0% saving, and needs 165 proofs instead of 374, which cuts off-chain proving from 6,096 s to 2,690 s. The figure is conservative: the 9 Tier-2 positions whose proof shows low risk submit no transaction

Table 6: Measured gas per transaction (real verifier and real proofs for Tier 2 and challenge).

`decideHeuristic` (bond as `msg.value` / from deposit)	269,534 / 274,772
`decideCommitment` (`msg.value` / deposit)	340,072 / 345,310
`revealCommitment`	39,089
`decideZkProof` (current / anchored features)	878,059 / 882,059
`anchorFeatures`	140,461
`challengeDecision` / `commitChallenge` + `revealChallenge`	678,771 / 94,771 + 678,838
`liquidate` (soft liquidation through the liquidator)	96,402
`reclaimBond` / `withdraw` / `deposit`	55,980 / 30,004 / 45,025
`currentFeatures` (view)	92,510
`registerModel` / `revokeModel` / `setTier2Verifier`	139,257 / 28,150 / 30,496
Losing racer (`PositionAlreadyFinalized`, live node)	26,149
Deploy: engine / verifier / demo pool / liquidator	3,286,971 / 3,422,602 / 847,742 / 201,232

Table 7: Gas of TrustTier vs always proving (M = million gas).

Dataset	N	T0/T1/T2	Tiered	All-zk	Saving
Real Aave V2	74	26/7/41	45.7 M	65.0 M	29.7%
Synthetic	300	100/76/124	164.6 M	263.4 M	37.5%
Combined	**374**	**126/83/165**	**210.3 M**	**328.4 M**	**36.0%**

Table 8: Ablations on the 374 positions (measured per-tier gas).

Variant	Saving	Cost of the change
TrustTier (deployed)	36.0%	–
No Tier-1 escalation	39.1%	21 false, slashable cheap claims
No Tier 1 (Tier 0 or Tier 2 only)	23.3%	Tier 1 is worth 12.7 points
Tier-1 band $B_1 = 0$	37.3%	7 claims a 5% drop breaches
Always Tier 0	69.3%	Nothing verified; 157 model-flagged positions unchecked

but are charged full gas. Always using Tier 0 would cost 100.8 M gas but verify nothing.

7.5 Ablations of the design

Table 8 removes one component at a time on the same 374 positions.

Escalation costs 3.1 points and buys honesty: without it, 21 cheap claims would contradict the router's own model (and, now that commitments must be opened, forfeit their bonds). Removing Tier 1 costs 12.7 points, which shows that the middle tier carries much of the saving. Section 7.6 explains the band rows.

How the design evolved. Table 9 follows the same measures through four versions. The midterm prototype was cheapest per decision because it checked almost nothing. Binding proofs to positions through an oracle-committed Poseidon hash fixed proof replay but added a 49,682-gas oracle transaction to every position and trusted the oracle. Verified inputs remove the oracle. Cheap tiers now pay for reading two feeds and storing a snapshot, while Tier 2 becomes cheaper: the proof has no in-circuit hash, and normalization shrank the circuit. The saving fell because the always-proving baseline also fell, but TrustTier's absolute cost is within 3.6% of the midterm's (210.3 M against 203.0 M), now with input integrity, tier enforcement, proof binding, escalation, and the incentive layer, which adds about 3 k gas per decision plus the Tier-1 reveal.

Table 9: Four versions of TrustTier on the same 374 positions.

	Midterm	Oracle	Verified	Final
Tier 0 gas	195,869	250,098	266,202	269,534
Tier 1 gas	286,356	340,550	336,713	379,161
Tier 2 gas	1,031,754	999,623	875,489	878,059
`verifyProof` gas	849,607	741,069	621,921	621,921
Proof bytes	5,696	4,480	3,584	3,584
Proving	25 s	20 s	16 s	16 s
Tiers (T0/T1/T2)	126/104/144	126/83/165	126/83/165	126/83/165
Saving	47.4%	39.9%	37.1%	36.0%
Inputs from	router	oracle	contract	contract
Tier enforced by	nobody	nobody	contract	contract
Slashing pays at	any	≤15 gwei	≤15 gwei	any
Liquidation executed	no	no	no	once, ≤30 min

7.6 Choosing the tier bands

The bands $B_0 = 0.15$ and $B_1 = 0.05$ are the two numbers TrustTier's guarantees hang on, so we asked how much the results depend on them. Replaying the 374 positions over 34 band settings (Figure 4) shows that both can be argued for rather than assumed.

The Tier-1 band is a safety margin. A cheap claim is honest only while the position stays below its threshold for the one-hour window, and a claim made at $\delta > B_1$ survives a sudden collateral price drop s whenever $B_1 \geq \theta s$. For $\theta = 0.80$ and a severe 5% hourly drop this requires $B_1 \geq 0.04$. The replay agrees: with $B_1 = 0$, seven cheap claims would be pushed past their threshold by such a drop; with $B_1 = 0.02$, three; with $B_1 = 0.05$, none.

The Tier-0 band trades gas for model oversight. With $B_1 = 0.05$, widening it from 0.08 to 0.30 lowers the saving from 38.7% to 33.9%, while the Tier-0 claims the model would flag fall from 5 to 2 at $B_0 = 0.10$, 1 at 0.15 and none from 0.20 (22 at $B_0 = 0.05$). Most of the safety is bought by 0.10; beyond it each step costs gas for little gain. The deployed $(0.15, 0.05)$ lies on the Pareto front of the grid over saving, flagged claims and exposed claims: no setting is better on all three. $B_0 = 0.10$ is an equally defensible choice that saves 1.6 more points for one more flagged claim.

The 0.80 threshold used for the synthetic stream matters less than it might appear. The contract never assumes it, and the real positions keep their own thresholds. Moving the synthetic threshold from 0.70 to 0.85 (Table 10) changes the saving from 23.9% to 38.1%, because the synthetic positions sit closer to or further from their boundary. The safety measures do not change: one flagged Tier-0 claim and no exposed claims at every threshold. Two caveats apply: "flagged" is the model's own verdict, not ground truth, and the 5% drop is a stress assumption, not a measured bound.

7.7 Model calibration and fidelity

The midterm model was trained on labels centred at LTV 0.5, below almost every position, so it flagged 84.1% of Tier-0 positions as high risk and made the Tier-0 check meaningless. Model v2 centres labels near the threshold and trains on normalized features: it flags 1

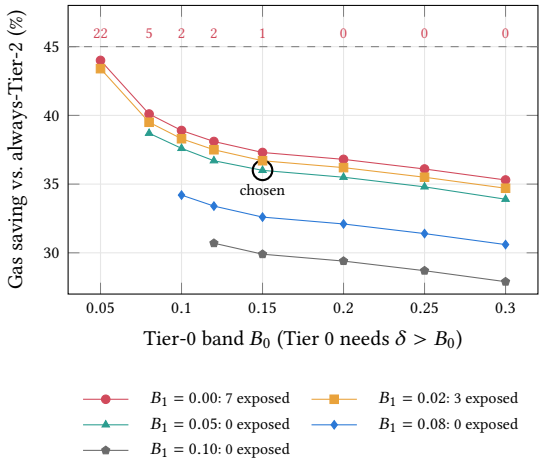


Figure 4: Gas saving over 374 positions as the tier bands vary. Red numbers along the top: Tier-0 claims the model would flag high risk at each B_0 (independent of B_1). "Exposed": cheap claims that a sudden 5% collateral price drop would push past the liquidation threshold. The circled point is the deployed setting ($B_0 = 0.15$, $B_1 = 0.05$).

Table 10: Sensitivity to the synthetic stream's liquidation threshold (bands fixed at 0.15/0.05; real positions keep their own thresholds).

Threshold	Tiers 0/1/2	Saving	Tier-0 flagged	Exposed
0.70	110/23/241	23.9%	1	0
0.75	126/44/204	30.0%	1	0
0.80	126/83/165	36.0%	1	0
0.85	163/52/159	38.1%	1	0

of 126 Tier-0 positions (0.8%), its validation accuracy is 95.9% (ROC-AUC 0.995), and the circuit agrees with the ONNX model on all 200 random inputs, so the proven label is the model's label. Of the 165 Tier-2 positions, 9 are scored low risk; their proof shows label 0, the engine would reject the liquidation, and the router submits nothing. Proximity to the boundary alone never liquidates a position; the proven model output does.

7.8 Against the health-factor rule

Is a model worth its gas compared with the rule every protocol already has? `evaluation/hf_baseline.py` simulates 3,000 positions (initial LTV 0.40–0.79, $\theta = 0.80$) for 30 days on identical price paths: a daily random walk with each position's own 30-day volatility, a common market factor (correlation 0.6) and a 2% daily chance of a market-wide crash of 12–30%. Debt is constant and borrowers never top up, a stress test rather than a forecast. Three policies act on the same paths. The *health-factor rule* liquidates 50% of the debt with a 5% bonus once LTV $\geq \theta$. *TrustTier* decides every position daily with its tiers and soft-liquidates (same close factor and bonus) when a proven model says high risk, with the health-factor rule as fallback. *Always zk* proves every position daily. The model runs through the same ONNX file the circuit is compiled from, on the contract's quantized features.

Table 11: 30-day stress test, 3,000 positions, identical price paths (seed 7). Bad debt and bonus as % of initial debt; gas is decision gas per position-day.

Policy	Forced	Insolv.	Bad debt	Bonus	Soft (unnec.)	Gas/day
HF rule	2,461	61	0.018%	1.35%	0	0
TrustTier	297	2	0.001%	1.67%	2,365 (323)	322 k
Always zk	297	2	0.001%	1.67%	2,368 (323)	880 k

Table 12: Cost of daily decisions per position (ETH = $3,000) and the smallest debt for which a year costs under 0.5% of it.

	TrustTier		Always zk	
Network	$/yr	Min. debt	$/yr	Min. debt
L1, 20 gwei	7,062	1.41 M	19,279	3.86 M
L1, 5 gwei	1,766	353 k	4,820	964 k
Rollup, 0.05 gwei	17.7	3.5 k	48.2	9.6 k

Table 11 shows the trade. Acting on the proven model cuts forced liquidations at the boundary by 88% and insolvencies from 61 to 2; over ten further scenarios the mean bad debt falls from 0.087% to 0.022% of debt and insolvencies from 198 to 45. The costs are real. Borrowers pay more liquidation bonus in total (1.67% against 1.35% of debt), because soft liquidations are more frequent though smaller, and 323 of the 1,264 soft-liquidated positions (26%) would never have reached the threshold. TrustTier gives the same protection as proving everything for 63% less decision gas, because its tiers prove only near the boundary.

7.9 Incentives and where TrustTier pays

`evaluation/bond_economics.py` prices the challenge game with measured gas. With the original fixed 0.01 ETH bond, slashing a false claim loses money above 14.7 gwei (12.9 gwei with commit-reveal): the verifier's dilemma at ordinary L1 prices. The gas-scaled bond pays a watcher at every gas price, for example +0.016 ETH at 20 gwei and +0.16 ETH at 200 gwei, and stays profitable while the base fee at challenge time is under 1.94× the claim's.

Decision gas also decides where TrustTier is worth deploying (Table 12). With one decision per position per day and a budget of 0.5% of the debt per year, daily monitoring on L1 only pays for positions above $353 k (5 gwei) to $1.4 M (20 gwei). On a rollup at an effective 0.05 gwei it pays from $3.5 k, ordinary retail size. TrustTier's natural home is therefore an L2 lending market, or L1 with a slower cadence for Tier-0 positions.

7.10 Real outcomes

The model's validation accuracy is measured on synthetic labels that are a formula of its own inputs, so it says little about real risk. `evaluation/outcome_labels.py` instead labels real Aave V2 snapshots by what happened: 1 if the health factor fell below 1 within 7 days. On the one wallet we hold (73 labelled snapshots, one positive) the deployed model reaches ROC-AUC 0.861 against 0.819 for a health-factor warning at 1.15, both with recall 1 and low precision. One wallet is a case study, not a statistic; the script runs unchanged

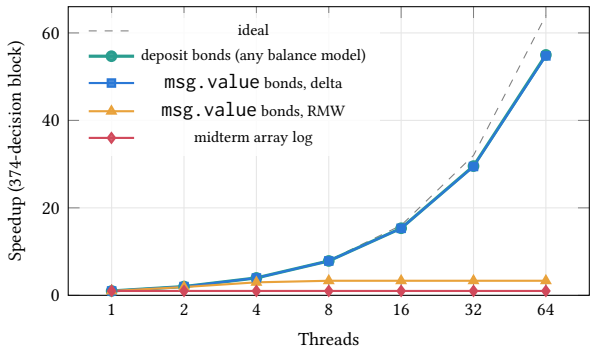


Figure 5: Pre-computed-schedule speedup of one traced block of 374 decisions. Deposit-funded bonds keep the block parallel whichever way an engine treats balance updates.

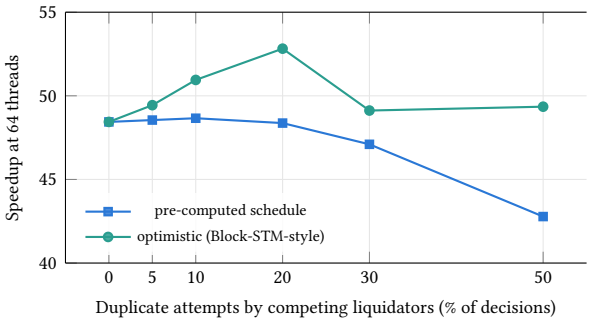


Figure 6: Same-position contention in traced blocks of 200 decisions plus duplicates. Losing attempts revert at the lock and conflict only with the winner, so the block still needs just 2 strict rounds; speedup degrades gently. Optimistic aborts at 64 threads rise from 0 to 49 (Table 13).

on Cred Protocol's complete 35,524-wallet dataset and retrains a candidate model on outcome labels with a time-ordered split, which is the evaluation we would cite next.

7.11 Concurrency

We study concurrency at three levels of realism: a modelled scheduler, as at the midterm; opcode-traced blocks executed by simulated parallel engines with 1 to 64 threads; and real competing liquidator threads on a live node.

Modelled scheduling. Modelling each decision's read/write set and scheduling a batch greedily into conflict-free rounds, the array-based audit log needs one round per decision (374 rounds for 374 decisions), and per-position storage needs one round at every batch size from 50 to 2,000 and every tier mix from 80/15/5 to 10/20/70. Modelled same-position contention of 5–50% raises this to 3–6 rounds.

Traced read/write sets. Models can miss shared state, so we measured it. For each scenario, `scripts/concurrency_trace.js` deploys the contracts with three markets, opens the positions, puts a whole batch of decisions from distinct liquidator accounts into *one block*, mines it, and replays every transaction with `debug_traceTransaction`. From the opcode trace we record every storage slot read (`SLOAD`) and written (`SSTORE`) in each contract, following the call stack, and every account balance moved. We then simulate three ways to execute the block, using each transaction's measured gas as its duration, with Tier 2 rescaled to the real-verifier cost: *strict rounds* (any read/write overlap conflicts); a *pre-computed schedule*, in which the block producer publishes a dependency graph that validators replay on T threads [3, 13]; and an *optimistic* Block-STM-style engine [17], in which T threads execute speculatively in block order, only read-after-write dependencies count, and a transaction that read a value a lower transaction later wrote is aborted and re-executed. Fees to the block producer are deferred, as in every parallel EVM. Bond transfers into the engine are modelled two ways: as a commutative balance delta, or as a read-modify-write of the engine's balance. Table 13 and Figures 5–6 report the results.

Five findings stand out.

(1) **Per-position storage parallelizes.** A block of 374 decisions has no shared storage write at all. It needs one strict round and reaches 15.3× on 16 threads and 54.8× on 64. Scaling falls short of ideal only because transaction durations differ: one Tier-2 decision costs 3.3 times a Tier-0 one, so the longest transactions bound the makespan. The midterm array log serializes the same block completely, whatever the engine.

(2) **The balance is the hidden hot spot.** Storage keys alone would suggest the final design is conflict-free. The traces show that every `msg.value` bond also writes the engine's ETH balance. Treated as a read-modify-write, that single location makes the 209 bonded claims a chain: 3.33× at any thread count, with 208 aborts in the optimistic engine. Deposit-funded bonds (Section 5.8) restore 15.3× under both balance models, for 5,238 gas per claim. As on historical Ethereum, conflicts concentrate on a little popular state [31]; tracing found ours where our model had not.

(3) **Racing liquidators cost little.** With 5–50% duplicate attempts, losing attempts revert at the per-position lock, write nothing, and conflict only with the winner. The block still needs only 2 strict rounds, and 64-thread speedup stays at 42.8–52.8×. Optimistic execution even beats the in-order schedule here: the schedule leaves a thread idle while it waits for a dependency, whereas the optimistic engine keeps every thread busy and pays only for a few aborts. The modelled scheduler had predicted 3–6 rounds, since it assumed losers write.

(4) **Oracle updates cause little conflict.** Chainlink heartbeat updates inside the block conflict with every later decision on that market, which reads the feed. Six updates raise strict rounds to 3 and cause 6 aborts at 16 threads, costing up to 2% of optimistic speedup at 16 threads and 13% at 64.

(5) **Parallelism needs many senders.** A single liquidator account submitting all 200 decisions serializes the block through its own nonce (1.0×). Parallel execution helps a market of liquidators, not one bot.

Competing liquidator threads on a live node. Finally, we ran K liquidator bots as real concurrent threads, each with its own account

Table 13: Parallel execution of traced blocks. "Delta": bond transfers are commutative balance deltas; "RMW": they read-modify-write the engine's ETH balance. Speedup = sequential gas / makespan for the pre-computed schedule (s) and the optimistic engine (o); Rounds: strict conflict-free rounds; Ab: aborts; Rev.: reverted.

Scenario (one block)	Bonds	Txs	Rev.	Rounds	16 thr s/o	64 thr s/o	Ab@16	Ab@64	Hottest shared write
Final design, 374 decisions	delta	374	0	1	15.32 / 15.32	54.75 / 54.75	0	0	none
Final design, 374 decisions	RMW	374	0	209	3.33 / 3.33	3.33 / 3.33	208	208	engine balance (209 txs)
Deposit-funded bonds, 374	either	374	0	1	15.34 / 15.34	54.97 / 54.97	0	0	none
Midterm array audit log, 374	either	374	0	374	1.00 / 1.00	1.00 / 1.00	373	373	log length slot (374 txs)
Final, 50 / 100 / 200 decisions	delta	–	0	1	13.2–15.0	31.4–48.4	0	0	none
Contention 5% (200 + 10 dups)	delta	210	10	2	15.00 / 15.00	48.55 / 49.44	0	4	none
Contention 20% (200 + 40)	delta	240	40	2	15.09 / 15.13	48.37 / 52.82	2	17	none
Contention 50% (200 + 100)	delta	300	100	2	14.57 / 15.15	42.78 / 49.35	11	49	none
Contention 20%	RMW	240	40	138	3.33 / 3.33	3.33 / 3.33	138	145	engine balance
1 / 3 / 6 oracle heartbeats	delta	201–206	0	2–3	15.0 / 14.6–14.8	47.3–48.5 / 42.3–45.7	4–6	16–27	price feeds
Tier mix 80/15/5, 10/20/70 (s = o)	delta	200	0	1	15.57, 15.12	43.58, 53.80	0	0	none
One liquidator sends all 200	delta	200	0	200	1.00 / 0.99	1.00 / 0.99	199	199	sender nonce

Table 14: K competing liquidator threads on a live node (30 positions, 1-second blocks).

Bots	Tries	Won	Reverted	Wasted gas/pos.	Once	Blocks
1	30	30	0	0	yes	1
2	60	30	30	26,149	yes	1
4	120	30	90	78,447	yes	2
8	240	30	210	183,043	yes	3
16	480	30	450	392,235	yes	6

Table 15: Prover pool on one 4-core machine: 12 real proofs per configuration.

Workers × threads	Proofs/min	Latency (s)	RSS/worker (MB)
1 × 4	9.25	6.5	1,761
2 × 2	9.97	12.0	1,590
4 × 1	10.46	22.8	1,514
Over HTTP, 2-core "machines": 1 → 5.05, 2 → 9.74 proofs/min			

and RPC connection, against a Hardhat node mining a block every second. Every bot tries to decide the same 30 positions (half Tier 0, half Tier 2) in its own random order and, like real racing bots, skips gas estimation, so losing transactions are mined and revert. Table 14 shows that every position was decided exactly once at every K (goal G6). Each losing attempt burns 26,149 gas, about a tenth of a Tier-0 decision, because the lock is checked before any external call; wasted gas grows linearly with the number of racers.

Limits of these results. Ethereum clients execute blocks sequentially today. Our speedups are what a parallel engine could achieve on TrustTier's real access patterns, with gas as a stand-in for execution time, not measured wall-clock time of a parallel client.

7.12 Proving throughput

Tier-2 proving is TrustTier's bottleneck, and it matters most in a crash, when many positions cross into Tier 2 at once. `evaluation/prover_pool.py` generates the same 12 real EZKL proofs under several configurations on a 4-core, 16 GB machine, after warming the workers up, and checks every proof's features and label. A single proof already parallelises well inside Halo2: 23.0 s on one thread, 11.8 s on two and 6.4 s on four (3.6×). Splitting one machine's cores among workers therefore trades latency for little throughput (Table 15): four single-thread workers prove 13% more per minute, but each proof takes 3.5 times longer, and every worker holds its own 1.5–1.8 GB copy of the proving key; oversubscribing the cores adds nothing. Scaling out is what works. With remote prover workers over HTTP, emulated as disjoint 2-core sets on the same host (so real machines can only do better), two "machines" prove 1.93× faster than one (5.05 → 9.74 proofs/min) at unchanged latency. For the 165 Tier-2 positions of the evaluation set, one 4-core machine needs 17.8 minutes; at the measured rate, four such machines would need about 3.9 (projected). On a live node, a 6-position batch through a 2-worker pool produced two real proofs in parallel and the real verifier accepted the approval (878,143 gas).

7.13 End-to-end validation

With the real verifier on a local node, the router opened and routed streams across three markets, producing real Tier-2 decisions at about 878,000 gas and ≈16 s of proving per position. Through the DApp, in a headless browser with an injected wallet, we opened a watch-band position (on-chain $\delta = 800$ bps, allowed tier 1, model: high risk), had the router claim Tier 1 dishonestly, generated a real proof on the claim's snapshot (16.2 s), and challenged from a second wallet. The bond was credited to the challenger, the position re-opened, and the same proof then approved its liquidation at Tier 2. Claiming Tier 0 on that position reverted with `TierNotAllowed(. . . , 0, 800)`. With the final contracts, the DApp's five-step guided demo runs the same story on a local node and then executes the soft liquidation through `TrustTierLiquidator`, taking δ from 8.0% to 22.1%; a browser test drives it end to end, on-chain and in simulation mode.

8 SECURITY ANALYSIS

We reviewed the code STRIDE-style and with an independent review pass; Table 16 condenses the 31 findings. We argue the goals of Section 4 as follows.

G1 Input integrity: every feature is computed by the contract from the position source and feeds within the transaction that uses it (finding 15). **G2 Tier soundness:** every cheap `decide*` recomputes δ and reverts below its band (finding 20). **G3 Proof binding:** `decideZkProof` compares the proof's five feature instances with its own computation and requires label 1, a registered model and the

verifier's own model (findings 9–11). **G4 Accountability:** a challenge must prove high risk on the claim's snapshot (finding 15). **G5 Liveness:** Tier 2 supersedes open claims, windows expire, and future timestamps clamp (finding 17). **G6 Exactly once:** the lock is set before any external call; tested on a live node with up to 16 racing threads. **G7 Governed change:** a multisig behind a timelock owns every contract (finding 3). **G8 Incentive compatibility:** gas-scaled bonds, commit-reveal challenges and must-open commitments (findings 8, 24, 25).

9 LIMITATIONS

Table 17 lists what remains and how each limitation is handled.

10 FUTURE WORK

Four directions follow from these results. *Batched settlement*: proving many Tier-2 decisions in one periodic proof would let a STARK's transparent setup and amortized verification win. *Data*: running the outcome-label evaluation on the full Cred Protocol dataset, and retraining on outcomes, would calibrate the model and the bands against ground truth. *L2 deployment*: measuring the scheme on a rollup, where Table 12 says it pays, including the sequencer check. *Parallel clients*: running the traced blocks on a parallel EVM would turn our simulated speedups into measured ones.

11 CONCLUSION

TrustTier began as a way to spend zero-knowledge proofs only where a liquidation decision is close enough to the boundary to need one. The idea worked: on 374 positions, tiering saves 36.0% of gas against always proving, and on identical stress paths a proven model prevents most of the forced liquidations and insolvencies the health-factor rule lets through. Tiering itself, however, created a party to trust, whoever chooses the tier and supplies the inputs. The final design removes that party. The contract reads its own inputs, enforces its own tiers, accepts only proofs about the state it sees, and judges cheap claims on the facts at the moment they were made. All of this costs little in absolute terms: 210.3 M gas against 203.0 M for the unprotected midterm design. Gas-scaled bonds, rewards, must-open commitments and commit-reveal challenges make watching worth someone's while, and the liquidator turns a verified decision into exactly one action. Measuring the system at the opcode level, rather than modelling it, also found the one shared write our model had missed. Deposit-funded bonds removed it, so a block of decisions now parallelizes to 15.3× on 16 threads. Risk-proportional verification on verified inputs is a pattern that should carry beyond liquidations, to credit scoring, insurance claims or any decision whose stakes vary.

ACKNOWLEDGMENTS

We thank the CSD436 course staff for review comments that shaped this work, in particular the observation that the chain could not check δ, which led to the verified-inputs design.

REFERENCES

[1] Aave. [n. d.]. Aave V3 Core: IPool Interface (getUserAccountData). https://github.com/aave/aave-v3-core/blob/master/contracts/interfaces/IPool.sol. Accessed October 2026.

[2] Abdulrahman Alhaidari, Balaji Palanisamy, and Prashant Krishnamurthy. 2025. On-Chain Decentralized Learning and Cost-Effective Inference for DeFi Attack Mitigation. In *7th Conference on Advances in Financial Technologies (AFT 2025) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 354)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 35:1–35:27. https://doi.org/10.4230/LIPIcs.AFT.2025.35

[3] Parwat Singh Anjana, Sweta Kumari, Sathya Peri, Sachin Rathor, and Archit Somani. 2019. An Efficient Framework for Optimistic Concurrent Execution of Smart Contracts. In *2019 27th Euromicro International Conference on Parallel, Distributed and Network-Based Processing (PDP)*. IEEE, 83–92. https://doi.org/10.1109/EMPDP.2019.8671637

[4] Parwat Singh Anjana, Sweta Kumari, Sathya Peri, Sachin Rathor, and Archit Somani. 2022. OptSmart: A Space Efficient Optimistic Concurrent Execution of Smart Contracts. *Distributed and Parallel Databases* (2022), 245–297. https://doi.org/10.1007/s10619-022-07412-y

[5] Nicola Atzei, Massimo Bartoletti, and Tiziana Cimoli. 2017. A Survey of Attacks on Ethereum Smart Contracts (SoK). In *Principles of Security and Trust (POST 2017) (Lecture Notes in Computer Science, Vol. 10204)*. Springer, 164–186. https://doi.org/10.1007/978-3-662-54455-6_8

[6] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. 2019. Scalable Zero Knowledge with No Trusted Setup. In *Advances in Cryptology – CRYPTO 2019 (Lecture Notes in Computer Science)*. Springer, 701–732. https://doi.org/10.1007/978-3-030-26954-8_23

[7] Vitalik Buterin. 2016. EIP-170: Contract Code Size Limit. Ethereum Improvement Proposals, https://eips.ethereum.org/EIPS/eip-170.

[8] Chainlink. [n. d.]. Getting Historical Data. Chainlink Data Feeds documentation, https://docs.chain.link/data-feeds/historical-data. Accessed October 2026.

[9] Wayne Chang, Gregory Rocco, Brantly Millegan, Nick Johnson, and Oliver Terbu. 2021. ERC-4361: Sign-In with Ethereum. Ethereum Improvement Proposals, https://eips.ethereum.org/EIPS/eip-4361.

[10] Bing-Jyue Chen, Suppakit Waiwitlikhit, Ion Stoica, and Daniel Kang. 2024. ZKML: An Optimizing System for ML Inference in Zero-Knowledge Proofs. In *Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys '24)*. ACM, 560–574. https://doi.org/10.1145/3627703.3650088

[11] KD Conway, Cathie So, Xiaohang Yu, and Kartin Wong. 2024. opML: Optimistic Machine Learning on Blockchain. arXiv:2401.17555. arXiv:2401.17555

[12] Philip Daian, Steven Goldfeder, Tyler Kell, Yunqi Li, Xueyuan Zhao, Iddo Bentov, Lorenz Breidenbach, and Ari Juels. 2020. Flash Boys 2.0: Frontrunning in Decentralized Exchanges, Miner Extractable Value, and Consensus Instability. In *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 910–927. https://doi.org/10.1109/SP40000.2020.00040

[13] Thomas Dickerson, Paul Gazzillo, Maurice Herlihy, and Eric Koskinen. 2017. Adding Concurrency to Smart Contracts. In *Proceedings of the ACM Symposium on Principles of Distributed Computing (PODC '17)*. ACM, 303–312. https://doi.org/10.1145/3087801.3087835

[14] Epic Web3. [n. d.]. Why Does StarkWare Use Block Packing? https://epicweb3.substack.com/p/why-does-starkware-use-block-packing. Accessed October 2026.

[15] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. 2019. PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge. IACR Cryptology ePrint Archive, Paper 2019/953. https://eprint.iacr.org/2019/953

[16] Sanjam Garg, Aarushi Goel, Somesh Jha, Saeed Mahloujifar, Mohammad Mahmoody, Guru-Vamsi Policharla, and Mingyuan Wang. 2023. Experimenting with Zero-Knowledge Proofs of Training. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS '23)*. ACM, 1880–1894. https://doi.org/10.1145/3576915.3623202

[17] Rati Gelashvili, Alexander Spiegelman, Zhuolun Xiang, George Danezis, Zekun Li, Dahlia Malkhi, Yu Xia, and Runtian Zhou. 2023. Block-STM: Scaling Blockchain Execution by Turning Ordering Curse to a Performance Blessing. In *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPoPP '23)*. ACM, 232–244. https://doi.org/10.1145/3572848.3577524

[18] Jens Groth. 2016. On the Size of Pairing-Based Non-interactive Arguments. In *Advances in Cryptology – EUROCRYPT 2016 (Lecture Notes in Computer Science, Vol. 9666)*. Springer. Full version: IACR Cryptology ePrint Archive, Paper 2016/260.

[19] Harry Kalodner, Steven Goldfeder, Xiaoqi Chen, S. Matthew Weinberg, and Edward W. Felten. 2018. Arbitrum: Scalable, Private Smart Contracts. In *27th USENIX Security Symposium (USENIX Security 18)*. USENIX Association, 1353–1370.

[20] Daniel Kang, Tatsunori Hashimoto, Ion Stoica, and Yi Sun. 2022. Scaling up Trustless DNN Inference with Zero-Knowledge Proofs. arXiv:2210.08674. arXiv:2210.08674

[21] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. 2010. Constant-Size Commitments to Polynomials and Their Applications. In *Advances in Cryptology – ASIACRYPT 2010 (Lecture Notes in Computer Science)*. Springer, 177–194. https://doi.org/10.1007/978-3-642-17373-8_11

Table 16: Security findings (STRIDE category) and status. Full write-ups are in `docs/SECURITY.md`.

#	Finding	Category	Severity	Status / fix
1	Push-payment DoS in bond payouts	DoS	High	Fixed: pull payments
2	Check ordering masked `AlreadyChallenged`	Info. disclosure	Low	Fixed
3	Single owner key	Elevation of privilege	High	Fixed: M-of-N multisig + timelock
4	Challenge label convention not per model	Tampering	Medium	Mitigated: verifier bound to one model
5	Self-generated SRS	Tampering	Medium	Documented: ceremony SRS for production
6	Public test key usable off local chains	Spoofing	Critical	Fixed: refused off local chains
7	Reentrancy	Tampering	–	Confirmed safe (CEI, pull payments)
8	Front-running of challenges	Tampering	Low	Fixed: commit-reveal challenges
9	Proof replay across positions	Tampering	Critical	Fixed: public features must equal on-chain q
10	Valid "low risk" proof approved a liquidation	Tampering	High	Fixed: label must be 1
11	Verifier not bound to the cited model	Spoofing	Medium	Fixed: `tier2ModelHash`
12	Unsalted Tier-1 commitment	Info. disclosure	Medium	Fixed: 32-byte salt
13	Duplicate revocation events	Repudiation	Low	Fixed
14	Out-of-range inputs hang EZKL	DoS	Medium	Fixed: clamp, full-domain calibration
15	Trust in a feature oracle	Spoofing	High	Fixed: features computed on-chain, snapshots
16	Timelock also delays emergency revocation	DoS	Low	Accepted trade-off
17	A cheap claim could lock or delay a position	DoS / Tampering	High	Fixed: lifecycle, Tier-2 supersedes
18	Router posted claims its model flags	Tampering	Medium	Fixed: escalation
19	Governance changes affect claims in their window	Tampering	Low	Accepted (timelocked)
20	The router, not the chain, chose the tier	Elevation of privilege	High	Fixed: δ and tiers enforced on-chain
21	Residual trust in protocol, feeds, source pointer	Spoofing	Medium	Accepted: explicit, staleness checks, timelocked
22	A proof can go stale while it is generated	DoS	Low	Fixed: feature anchors (10 min, never safer)
23	`msg.value` bonds share one balance	DoS (throughput)	Low	Fixed: deposit-funded bonds
24	Fixed bond: challenging unprofitable above 14.7 gwei	DoS (incentives)	High	Fixed: bond scales with base fee; reward pool
25	Tier-1 commitment never checked on-chain	Repudiation	Medium	Fixed: must be opened in the window
26	Approvals had no consumer and never expired	Elevation of privilege	Medium	Fixed: liquidator consumes once, 30-min TTL
27	Chainlink phase boundary made a round unusable	DoS	Low	Fixed: heartbeat-bounded fallback
28	Stale L2 prices after a sequencer outage	Tampering	Medium	Fixed: optional uptime feed + 1 h grace
29	Cheap claims emitted `LiquidationApproved`	Repudiation	Low	Fixed: `SafeClaimRecorded`
30	An RPC proxy would expose the node's unlocked accounts	Elevation of privilege	Critical	Fixed: method allow-list
31	Browser-generated keys live in browser storage	Info. disclosure	Medium	Mitigated: encrypted keystore, testnet only

Table 17: Remaining limitations.

Limitation	Impact and handling
Self-generated KZG SRS	Soundness depends on the setup's secret being destroyed; use a ceremony SRS or a STARK for production
Oracle manipulation [33]	A manipulated feed moves q and δ; staleness checks only; TWAP or decentralized oracles are the protocol's responsibility
Volatility feeds	Not available on every network; a mock volatility feed is used on testnets
Aave account history	The adapter counts age from registration and records top-ups when poked
Real data	One Aave V2 wallet (74 snapshots); the outcome-label pipeline is ready for the full dataset but was not run on it
Model labels	Synthetic; "flagged" measures agreement with the model, not ground truth
Unnecessary liquidations	26% of soft-liquidated positions would have recovered in the stress test
L1 economics	Daily decisions pay only for large positions on L1; the target is a rollup
Concurrency	Simulated parallel engines on traced sets; no parallel EVM client was run
Base-fee spikes	A bond covers challenges while the base fee stays under 1.94× its value at the claim
Public deployment	One-click Sepolia workflow; needs a funded testnet key as a repository secret

[22] Haoran Lin, Hang Feng, Yajin Zhou, and Lei Wu. 2025. ParallelEVM: Operation-Level Concurrent Transaction Execution for EVM-Compatible Blockchains. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys '25)*. ACM. https://doi.org/10.1145/3689031.3696063

[23] Ayush Nainwal, Atharva Kamble, and Nitin Awathare. 2025. A Comparative Analysis of zk-SNARKs and zk-STARKs: Theory and Practice. arXiv:2512.10020. arXiv:2512.10020

[24] OpenZeppelin. [n. d.]. Governance: TimelockController (OpenZeppelin Contracts 5.x). https://docs.openzeppelin.com/contracts/5.x/api/governance. Accessed October 2026.

[25] Zhizhi Peng, Chonghe Zhao, Taotao Wang, Guofu Liao, Zibin Lin, Yifeng Liu, Bin Cao, Long Shi, Qing Yang, and Shengli Zhang. 2025. A Survey of Zero-Knowledge Proof Based Verifiable Machine Learning. arXiv:2502.18535. arXiv:2502.18535

[26] Daniel Perez, Sam M. Werner, Jiahua Xu, and Benjamin Livshits. 2021. Liquidations: DeFi on a Knife-Edge. In *Financial Cryptography and Data Security (FC 2021) (Lecture Notes in Computer Science)*. Springer, 457–476. https://doi.org/10.1007/978-3-662-64331-0_24

[27] Manaswini Piduguralla, Souvik Sarkar, Arunmoezhi Ramachandran, and Sathya Peri. 2026. BlockRaFT: A Distributed Framework for Fault-Tolerant and Scalable Blockchain Nodes. arXiv:2604.15731; to appear at IEEE ICSA 2026. https://arxiv.org/abs/2604.15731

[28] Kaihua Qin, Liyi Zhou, Pablo Gamito, Philipp Jovanovic, and Arthur Gervais. 2021. An Empirical Study of DeFi Liquidations: Incentives, Risks, and Instabilities. In *Proceedings of the 21st ACM Internet Measurement Conference (IMC '21)*. ACM, 336–350. https://doi.org/10.1145/3487552.3487811

[29] Kaihua Qin, Liyi Zhou, and Arthur Gervais. 2022. Quantifying Blockchain Extractable Value: How Dark Is the Forest?. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 198–214. https://doi.org/10.1109/SP46214.2022.9833734

[30] RISC Zero. [n. d.]. Recursion: STARK-to-SNARK Compression. RISC Zero developer documentation, https://dev.risczero.com/api/recursion. Accessed October 2026.

[31] Vikram Saraph and Maurice Herlihy. 2020. An Empirical Study of Speculative Concurrency in Ethereum Smart Contracts. In *International Conference on Blockchain Economics, Security and Protocols (Tokenomics 2019) (OpenAccess Series in Informatics (OASIcs), Vol. 71)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 4:1–4:15. https://doi.org/10.4230/OASIcs.Tokenomics.2019.4

[32] Atefeh Zareh Chahoki, Maurice Herlihy, and Marco Roveri. 2025. Conthereum: Concurrent Ethereum Optimized Transaction Scheduling for Multi-Core Execution. arXiv:2504.07280. arXiv:2504.07280

[33] Liyi Zhou, Xihan Xiong, Jens Ernstberger, Stefanos Chaliasos, Zhipeng Wang, Ye Wang, Kaihua Qin, Roger Wattenhofer, Dawn Song, and Arthur Gervais. 2023. SoK: Decentralized Finance (DeFi) Attacks. In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2444–2461. https://doi.org/10.1109/SP46215.2023.10179435

[34] Zkonduit. [n. d.]. EZKL: Zero-Knowledge Inference for Deep Learning Models (version 23.0.5). https://github.com/zkonduit/ezkl. Accessed October 2026.

[35] zkSecurity. [n. d.]. Verifying Cairo Proofs on Ethereum. https://blog.zksecurity.xyz/posts/stark-evm-adapter/. Accessed October 2026.

A REPRODUCING THE RESULTS

All numbers in this paper are produced by scripts in the project repository.

- Tests: `npx hardhat test` (126) and `python -m pytest off-chain/tests` (92).
- Prover pool: `python -m evaluation.prover_pool` (needs `circuits/pk.key`).
- Health-factor baseline, bond economics, real outcomes: `python -m evaluation.hf_baseline`, `evaluation.bond_economics`, `evaluation.outcome_labels`.
- Gas: `npx hardhat run scripts/measure_gas.js`.
- Circuit and proofs: `python3 circuits/build_circuit.py`; proving key only: `circuits/make_proving_key.py`.
- Cost, ablations, bands: `python -m evaluation.evaluate`, `evaluation.band_sensitivity`.
- Modelled concurrency: `evaluation.concurrency_sim`, `evaluation.concurrency_ablation`.
- Traced concurrency: `npx hardhat run scripts/concurrency_trace.js`, then `python -m evaluation.parallel_exec`.
- Liquidator race: `npx hardhat node`, a mock-verifier deployment, then `python -m evaluation.liquidator_race`.